



Politechnika Wrocławska

Struktury danych

Analiza porównawcza wydajności list wiązanych i tablic dynamicznych w języku C++

Piotr Frąc

Marzec 2026

1 Wprowadzenie

Projekt skupia się na analizie porównawczej dwóch fundamentalnych struktur liniowych: *linked list* i *dynamic array*

Celem pracy jest implementacja obu struktur w języku C++ oraz przeprowadzenie testów wydajnościowych dla podstawowych operacji, takich jak:

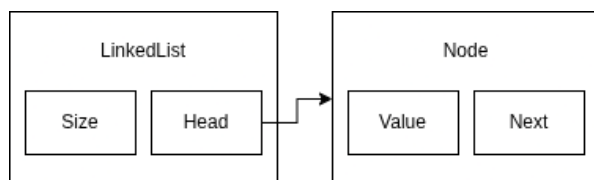
- dodawanie i usuwanie elementów (na początku, na końcu oraz na zadanym indeksie),
- wyszukiwanie elementów o określonej wartości,
- dostęp do elementu poprzez indeks.

Badanie ma na celu wykazanie różnic w złożoności obliczeniowej oraz praktycznym czasie wykonania, wynikających ze specyfiki alokacji pamięci. Pełny kod źródłowy projektu wraz z historią zmian dostępny jest w repozytorium pod adresem: https://github.com/Elrrior-programmer/Struktury-Danych/tree/main/P1%20-%20Lista_wizana

1.1 Zaimplementowane struktury danych

Wszystkie struktury danych zostały wykonane zgodnie z wykładami i prezentacjami dr inż. Radosława Rudego, które są dostępne pod linkiem: <https://kam.pwr.edu.pl/jaroslawn-rudy/#tab-struktury-danych-wyklad>

Lista wiązana



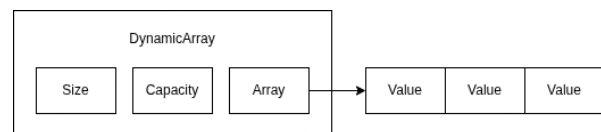
Zaimplementowana **lista** jest przykładem *Listy jednokierunkowej liniowej (Singly Linked List)*, która charakteryzuje się:

- **Jednokierunkowość:** Każdy węzeł (*node*) przechowuje wskaźnik wyłącznie do swojego następcy. Oznacza to,

że przeglądanie struktury możliwe jest tylko od głowy (*head*) w stronę końca listy.

- **Liniowość (brak cyliczności):** Struktura posiada wyraźnie zdefiniowany początek oraz koniec. Ostatni element przechowuje wartość pustą (`nullptr`), co stanowi jednoznacznie zakończenie sekwencji danych.

Tablica Dynamiczna



Zaimplementowana **tablica dynamiczna** charakteryzuje się specyficznymi usprawnieniami implementacyjnymi:

- **Inicjalizacja niezerowa:** Tablica rozpoczyna pracę z domyślną pojemnością *capacity* = 4, co eliminuje potrzebę re-alokacji przy pierwszych operacjach dodawania.
- **Progresja geometryczna** ($2 \times capacity$): W przypadku przepełnienia bufora, jego rozmiar jest zwiększany dwukrotnie. Zapewnia to zamortyzowany koszt dopisania elementu na poziomie $O(1)$.
- **Redukcja pamięci** ($capacity/2$): Gdy liczba elementów spadnie do poziomu $size \leq capacity/4$, rozmiar tablicy jest zmniejszany o połowę.

2 Metodyka i środowisko badawcze

W tej sekcji opisano parametry techniczne stanowisk testowych oraz metodologię przeprowadzenia eksperymentów wydajnościowych.

2.1 Narzędzia pomiarowe

Do precyzyjnego wyznaczenia czasu trwania operacji wykorzystano bibliotekę `<chrono>` oraz zegar o wysokiej rozdzielczości `std::chrono::high_resolution_clock`. Wybór ten pozwolił na rejestrację różnic czasowych rzędu nanosekund, co jest kluczowe przy analizie struktur danych dla małych wartości n .

2.2 Specyfikacja stanowiska badawczego

Do przeprowadzenia testów porównawczych wykorzystano dwie maszyny o skrajnie różnych architekturach, co pozwala na ocenę wydajności struktur zarówno na nowoczesnym sprzęcie typu high-end, jak i na starszych jednostkach budżetowych

Stanowisko 1 (Wydajne):

- **Procesor:** Intel Core i7-14700KF
- **Słowo Procesorowe:** 64 Bity
- **RAM:** 32,0 GB DDR5 (6400 MT/s)
- **OS:** Windows 11 Home
- **Kompilator:** g++.exe v15.2.0

Stanowisko 2 (Starsza architektura):

- **Procesor:** Intel Core i5-620
- **Słowo Procesorowe:** 64 Bity
- **RAM:** 8,0 GB DDR3 (1600 MT/s)
- **OS:** Kali GNU/Linux 2026.1
- **Kompilator:** g++ v15.2.0-15

2.3 Zróżnicowanie rozmiaru danych

W badaniach wykorzystano typy danych o różnych rozmiarach w celu analizy wpływu

szerokości słowa procesorowego na wydajność struktur:

- **Small (1 B):** Typ `char` – dane wielokrotnie mniejsze od słowa procesorowego.
- **Word-aligned (8 B):** Typ `long long` – rozmiar odpowiadający natywnemu słowu architektury x86-64.
- **Medium (24 B):** Struktura złożona z trzech słów – wymaga wielokrotnego dostępu do pamięci dla jednego obiektu.
- **Large (256 B):** Duży obiekt testujący koszt kopiowania danych przy realokacji tablicy oraz zalety przepinania wskaźników w liście.

2.4 Wykonane eksperymenty

Dla każdej struktury danych przeprowadzono trzy rodzaje eksperymentów, powtarzając je dla wszystkich czterech typów danych (`char`, `long long`, `Medium 24B`, `Large 256B`).

- **Dodawanie elementów** dla trzech wariantów wstawiania: na początku struktury, na środku (indeks $n/2$) oraz na końcu.
- **Usuwanie elementów** analogicznie do dodawania, usuwanie badano w trzech miejscach: z początku, ze środka (indeks $n/2$) oraz z końca struktury.
- **Wyszukiwanie pierwszego elementu k** w strukturze o wielkości miliona

Metodyka pomiarów

Każdy eksperyment powtórzono 10-krotnie, a wynik końcowy stanowi średnią arytmetyczną z tych prób. Pozwala to zminimalizować wpływ zakłóceń systemowych. Pomiaru wykonywano niezależnie na obu stanowiskach badawczych opisanych w sekcji 2.2.

3 Wyniki i wnioski

sprawdzam ile liniej do nowej strony
sprawdzam ile liniej do nowej strony